DD2552 Seminar 3: Simply Typed Lambda Calculus and Beyond

Karl Palmskog

KTH

Wednesday September 6, 2023

KTH Research Preparation course

Module D: PL Semantics

Pablo Buiras

Course material

- PFPL chapter 4
- Commentary in Software Foundations
  - https://softwarefoundations.cis.upenn.edu/plf-current/Stlc.html
  - https://softwarefoundations.cis.upenn.edu/plf-current/StlcProp.html

λ-calculus with evaluation order

CBV

Call-by-value (CBV)
λ-calculus

(applicative order)

Syntax | Inductive set

$x, y, z, \ldots$ variables

$M, N ::= x$ terms
$\quad\quad | \; M\ N$
$\quad\quad | \; \lambda x.M$

$v ::= \lambda x.M$ values

Operational semantics | Inductive relation

$$\frac{}{(\lambda x.M)v \to_\beta M[v/x]}\ \beta$$

$$\frac{M \to_\beta M'}{M\ N \to_\beta M'\ N}$$

$$\frac{N \to_\beta N'}{v\ N \to_\beta v\ N'}$$

Properties | Proofs

Church-Rosser (confluence) Equational theory
Turing completeness Fixed-point existence

Extensions

Natural numbers & addition

x, y, z, ...	variables
M, N ::= x	terms
| M N	
| λx.M	
v ::= λx.M	values

x, y, z, ...	variables
n, m, ...	literals
M, N ::= x	terms
| n	
| add(M,N)	
| M N	
| λx.M	
v ::= λx.M	values
| n	

Extensions – δ rules

Natural numbers & addition

Literal addition

$$\frac{r = m+n}{\mathrm{add}(m,n) \rightarrow r}$$

Context closure

$$\frac{M \rightarrow M'}{\mathrm{add}(M,N) \rightarrow \mathrm{add}(M',N)} \qquad \frac{N \rightarrow N'}{\mathrm{add}(v,N) \rightarrow \mathrm{add}(v,N')}$$

Extensions – δ rules

Booleans

Syntax

x, y, z, ...	variables
M, N, R ::= x	terms
| true | false	
| if M then N else R	
| M N	
| λx:τ.M	
v ::= λx:τ.M	values
| true | false	

Operational semantics

$$\frac{}{(\lambda x.M)v \to M[v/x]}\ \beta$$

$$\frac{}{\text{if true then N else R} \to \text{N}}\ \text{If-true}$$

$$\frac{}{\text{if false then N else R} \to \text{R}}\ \text{If-false}$$

+ context closure rules

Programs can go wrong

- Reduction not always leads to a value
  - Nontermination
  - Non-value normal forms often make no sense
- Problem gets worse as we keep adding extensions
  - add(1,true) → ?
  - if (λx.x) then λy.true else false → ?

Type systems

- **Idea**: classify terms by a set of syntax-based rules
- Terms that ‘make sense’ are assigned a **type** (well-typed)
- Terms with the same type have related semantics/behaviour
- Terms that do not have a type are not supposed to be evaluated
- Formally, we have a set of types T, a set of environments Env = Var → T, and a typing judgment relation

$$_\vdash_:_ \subseteq \text{Env} \times \Lambda \times T$$

$$\Gamma \vdash 1 : \text{nat}$$
$$\Gamma \vdash \mathtt{true} : \text{bool}$$
$$\Gamma \vdash \mathtt{add}(1,2) : \text{num}$$

Why types?

- lack of restrictions in formal systems can lead to contradictions (paradoxes)
- types can enforce enough discipline to rule out contradictions
- "set of all sets" vs. "class of all sets" vs. hierarchy of classes

Why Simply Typed Lambda Calculus?

- possibly simplest meaningful typesystem for lambda calculus (add function types)
- showcases why we have (several) types in functional languages
- stepping stone to practical languages like CakeML
- usually abbreviated STLC

What is STLC?

- lambda calculus with a typing relation
- a basis for metatheory (safety and progress)
- benchmark for formal metatheory

Lambda Calculus syntax

t	$::=$			term
	$\mid$	x		variable
	$\mid$	$\lambda x.t$	bind x in t	lambda
	$\mid$	$t\,t'$		app
	$\mid$	(t)	S	
	$\mid$	$[t/x]t'$	M	
v	$::=$			value
	$\mid$	$\lambda x.t$		lambda
$typ,\ T$	$::=$			types
	$\mid$	o		base type
	$\mid$	$T_1 \to T_2$		function types

Lambda Calculus reduction reminder

$$\frac{}{(\lambda x.t_{12})\ v_2 \longrightarrow [v_2/x]t_{12}} \quad \text{RED__AX__APP}$$

$$\frac{t_1 \longrightarrow t_1'}{t_1\ t \longrightarrow t_1'\ t} \quad \text{RED__CTX__APP__FUN}$$

$$\frac{t_1 \longrightarrow t_1'}{v\, t_1 \longrightarrow v\, t_1'} \quad \text{RED__CTX__APP__ARG}$$

STLC typing relation

$$\frac{x : T \in \Gamma}{\Gamma \vdash x : T} \quad \text{TYPING_VAR}$$

$$\frac{\Gamma, x : T_1 \vdash t : T_2}{\Gamma \vdash \lambda x.t : T_1 \to T_2} \quad \text{TYPING_ABS}$$

$$\frac{\begin{array}{l}\Gamma \vdash t_1 : T_1 \to T_2 \\ \Gamma \vdash t_2 : T_1\end{array}}{\Gamma \vdash t_1\ t_2 : T_2} \quad \text{TYPING_APP}$$

Instantiating STLC with booleans

t	$::=$			term
	$\mid$	x		variable
	$\mid$	$\lambda x.t$	bind x in t	lambda
	$\mid$	$t\,t'$		app
	$\mid$	$\mathbf{if}\,t\,\mathbf{then}\,t'\,\mathbf{else}\,t''$		conditional
	$\mid$	true		true
	$\mid$	false		false
	$\mid$	(t)	S	
	$\mid$	$[t/x]t'$	M	
v	$::=$			value
	$\mid$	$\lambda x.t$		lambda
$typ,\ T$	$::=$			types
	$\mid$	Bool		bool type
	$\mid$	$T_1 \to T_2$		function types

Instantiating STLC with booleans

$$\frac{}{(\lambda x.t_{12})\ v_2 \longrightarrow [v_2/x]t_{12}} \quad \text{RED_AX_APP}$$

$$\frac{t_1 \longrightarrow t_1'}{t_1\ t \longrightarrow t_1'\ t} \quad \text{RED_CTX_APP_FUN}$$

$$\frac{t_1 \longrightarrow t_1'}{v\ t_1 \longrightarrow v\ t_1'} \quad \text{RED_CTX_APP_ARG}$$

$$\frac{}{\textbf{if}\ \mathsf{true}\ \textbf{then}\ t_1\ \textbf{else}\ t_2 \longrightarrow t_1} \quad \text{RED_IF_TRUE}$$

$$\frac{}{\textbf{if}\ \mathsf{false}\ \textbf{then}\ t_1\ \textbf{else}\ t_2 \longrightarrow t_2} \quad \text{RED_IF_FALSE}$$

$$\frac{t_1 \longrightarrow t_1'}{\textbf{if}\ t_1\ \textbf{then}\ t_2\ \textbf{else}\ t_3 \longrightarrow \textbf{if}\ t_1'\ \textbf{then}\ t_2\ \textbf{else}\ t_3} \quad \text{RED_IF}$$

Instantiating STLC with booleans

$$\frac{x : T \in \Gamma}{\Gamma \vdash x : T} \quad \text{TYPING_VAR}$$

$$\frac{\Gamma, x : T_1 \vdash t : T_2}{\Gamma \vdash \lambda x.t : T_1 \to T_2} \quad \text{TYPING_ABS}$$

$$\frac{\begin{array}{l}\Gamma \vdash t_1 : T_1 \to T_2 \\ \Gamma \vdash t_2 : T_1\end{array}}{\Gamma \vdash t_1\ t_2 : T_2} \quad \text{TYPING_APP}$$

$$\frac{}{\Gamma \vdash \mathsf{true} : \mathsf{Bool}} \quad \text{TYPING_TRUE}$$

$$\frac{}{\Gamma \vdash \mathsf{false} : \mathsf{Bool}} \quad \text{TYPING_FALSE}$$

$$\frac{\begin{array}{l}\Gamma \vdash t_1 : \mathsf{Bool} \\ \Gamma \vdash t_2 : T_1 \\ \Gamma \vdash t_3 : T_1\end{array}}{\Gamma \vdash \mathbf{if}\ t_1\ \mathbf{then}\ t_2\ \mathbf{else}\ t_3 : T_1} \quad \text{TYPING_IF}$$

Example typing derivations

$$\dfrac{\dfrac{x : \text{bool} \in \Gamma'}{\Gamma' \vdash x : \text{bool}} \qquad \dfrac{\dfrac{y : \text{bool} \in \Gamma'}{\Gamma' \vdash y : \text{bool}} \quad \Gamma' \vdash \text{true} : \text{bool} \quad \Gamma' \vdash \text{false} : \text{bool}}{\Gamma' \vdash \text{if } y \text{ then true else false} : \text{bool}} \qquad \dfrac{}{\Gamma' \vdash \text{false} : \text{bool}}}{\dfrac{\Gamma, x{:}\text{bool}, y{:}\text{bool} \vdash \text{if } x \text{ then (if } y \text{ then true else false) else false} : \text{bool}}{\dfrac{\Gamma, x{:}\text{bool} \vdash (\lambda y{:}\text{bool}.\ \text{if } x \text{ then (if } y \text{ then true else false) else false}) : \text{bool} \to \text{bool}}{\Gamma \vdash (\lambda x{:}\text{bool}.\ \lambda y{:}\text{bool}.\ \text{if } x \text{ then (if } y \text{ then true else false) else false}) : \text{bool} \to \text{bool} \to \text{bool}}}}$$

Properties

Canonical forms

- Well-typed values are of a specific form that depends on their type.
- **Canonical Forms Lemma:** Assume M is a value.
  - $\Gamma \vdash M : \sigma \to \tau \Rightarrow M = \lambda x{:}\sigma.N$ for some x, N.
  - $\Gamma \vdash M : \mathrm{bool} \Rightarrow M = \mathrm{true}$ or $M = \mathrm{false}$
- **Proof.** Trivial by inspection of the typing rules and definition of values.

STLC property: progress

Pierce et al.:

> *"closed, well-typed terms are not stuck: either a well-typed term is a value, or it can take a reduction step."*

Theorem

For all terms t and types T, if $\bullet \vdash t : T$, then t is a value or there exists t' such that $t \longrightarrow t'$.

Progress

- If $\vdash M : \sigma$, either M is a value or $\exists N. M \to N$
- **Proof.** Induction on typing derivation
  - (Var): $\vdash x : \sigma$, not possible
  - (Bool-1, Bool-2, Abs): Trivial
    - $\vdash$ true, false : bool or $\vdash \lambda x{:}\tau.M : \tau \to \sigma$
  - (App): $\vdash M\ N : \sigma$, and $\vdash M : \tau \to \sigma$, $\vdash N : \tau$
    - By IH, either M is a value or it can take a step. (cont)

Progress (2)

- Case M is a value: Now by IH then N is either a value or can take a step.
  - Case N is a value: By canonical forms lemma, M is an abstraction, so we have a redex.
  - Case N can take a step: M N can take a step by context closure.
- Case M can take a step: M N can take a step by context closure.

– (If): by a similar case analysis after applying IH.

STLC property: preservation

Pierce et al.:

> *if a closed, well-typed term t has type T and takes a step to t', then t' is also a closed term with type T. In other words, the small-step reduction relation preserves types.*

Theorem

For all terms t, t' and types T, if $\bullet \vdash t : T$ and $t \longrightarrow t'$, then $\bullet \vdash t' : T$.

Preservation

- If $\Gamma \vdash M : \sigma$ and $M \to N$ then $\Gamma \vdash N : \sigma$
- **Proof.** By induction on the derivation of $M \to N$.
  - Case (β): $M = (\lambda x{:}\tau.M_1)\ M_2$, hence

    $N = M_1[M_2/x]$. Result follows by Subst. lemma (omitted).
  - Case context rules: Similar result, typing is closed under contexts (omitted).
  - Case (If-true): M = if true then M_1 else M_2, $N = M_1$.

    From $\Gamma \vdash M : \sigma$ we know $\Gamma \vdash M_1 : \sigma$.
  - Case (If-false): analogous

Properties

Type system Soundness

- Soundness = progress + preservation
- Progress:
  - If $\vdash M : \sigma$, either M is a value or $\exists N.M \rightarrow N$
- Preservation:
  - If $\Gamma \vdash M : \sigma$ and $M \rightarrow N$ then $\Gamma \vdash N : \sigma$

Properties

- Normal forms and nontermination
  - Ω is not well-typed
  - (Weak) Normalisation: All well-typed terms have a normal form
- Fixed points?
  - Y is not well-typed

Fixed points

M, N, R ::=	x	terms
	| M N	
	| true | false	
	| if M then N else R	
	| $\lambda x{:}\tau.M$	
	| fix M	

$$\frac{\Gamma \vdash M : \tau \to \tau}{\Gamma \vdash \text{fix } M : \tau} \quad \text{ty-fix}$$

$$\frac{}{\text{fix } M \to M\ (\text{fix } M)} \quad \text{fix}$$

Russell's Paradox

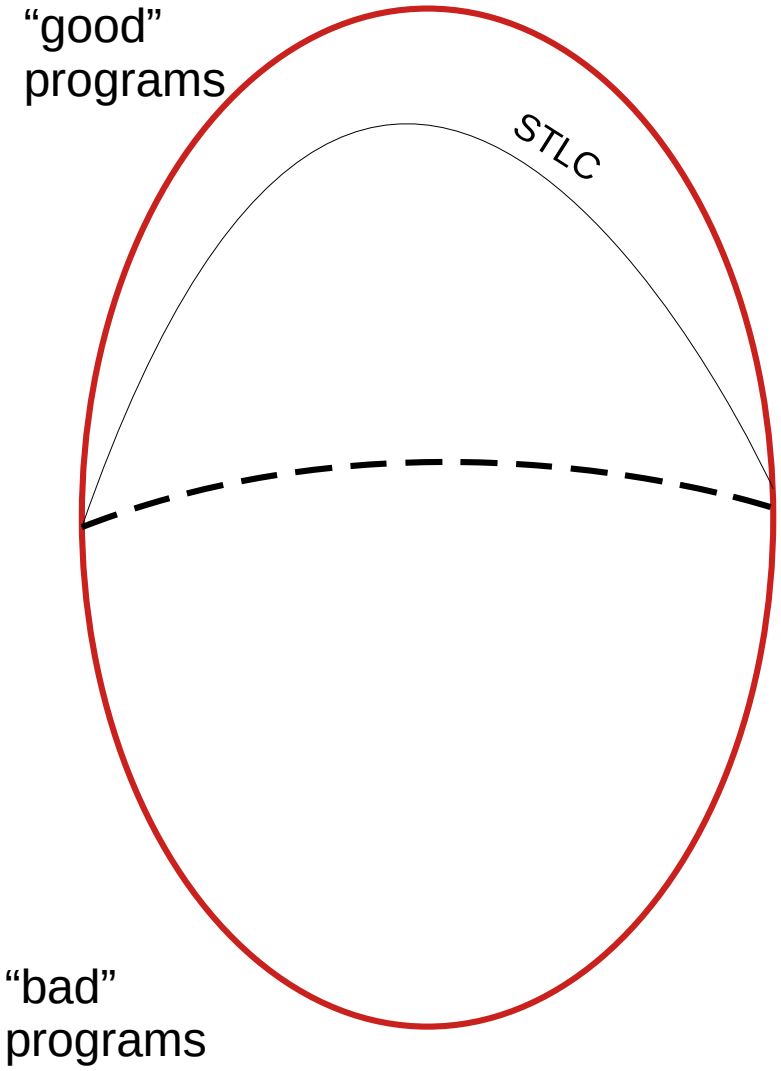
- Y can be seen as an instance of Russell's paradox
- Consider sets represented by characteristic functions
  - $g = \lambda x{:}bool.\ \neg(x\ x)$

    "the set of all sets that don't contain themselves"
  - $g\ g$?
    - $g\ g =_\beta \neg(g\ g)$

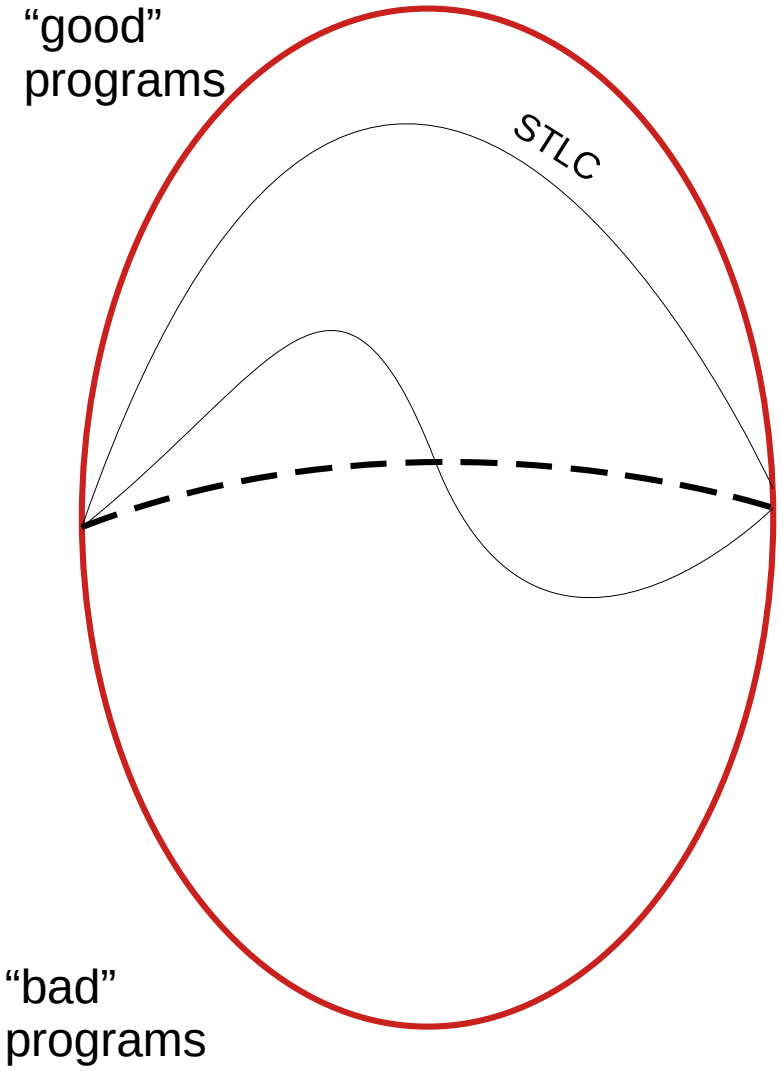
Typing Ω fails

$$
\dfrac{\dfrac{\dfrac{\dfrac{?}{\Gamma, x{:}bool \vdash x : \tau \to \sigma} \quad \Gamma, x{:}bool \vdash x : \tau}{\Gamma, x{:}bool \vdash x\ x : \sigma}}{\Gamma \vdash (\lambda x{:}bool.x\ x) : bool \to \sigma} \quad \Gamma \vdash (\lambda x{:}bool.x\ x) : bool}{\Gamma \vdash (\lambda x{:}bool.x\ x)\ (\lambda x{:}bool.x\ x) : \sigma}
$$

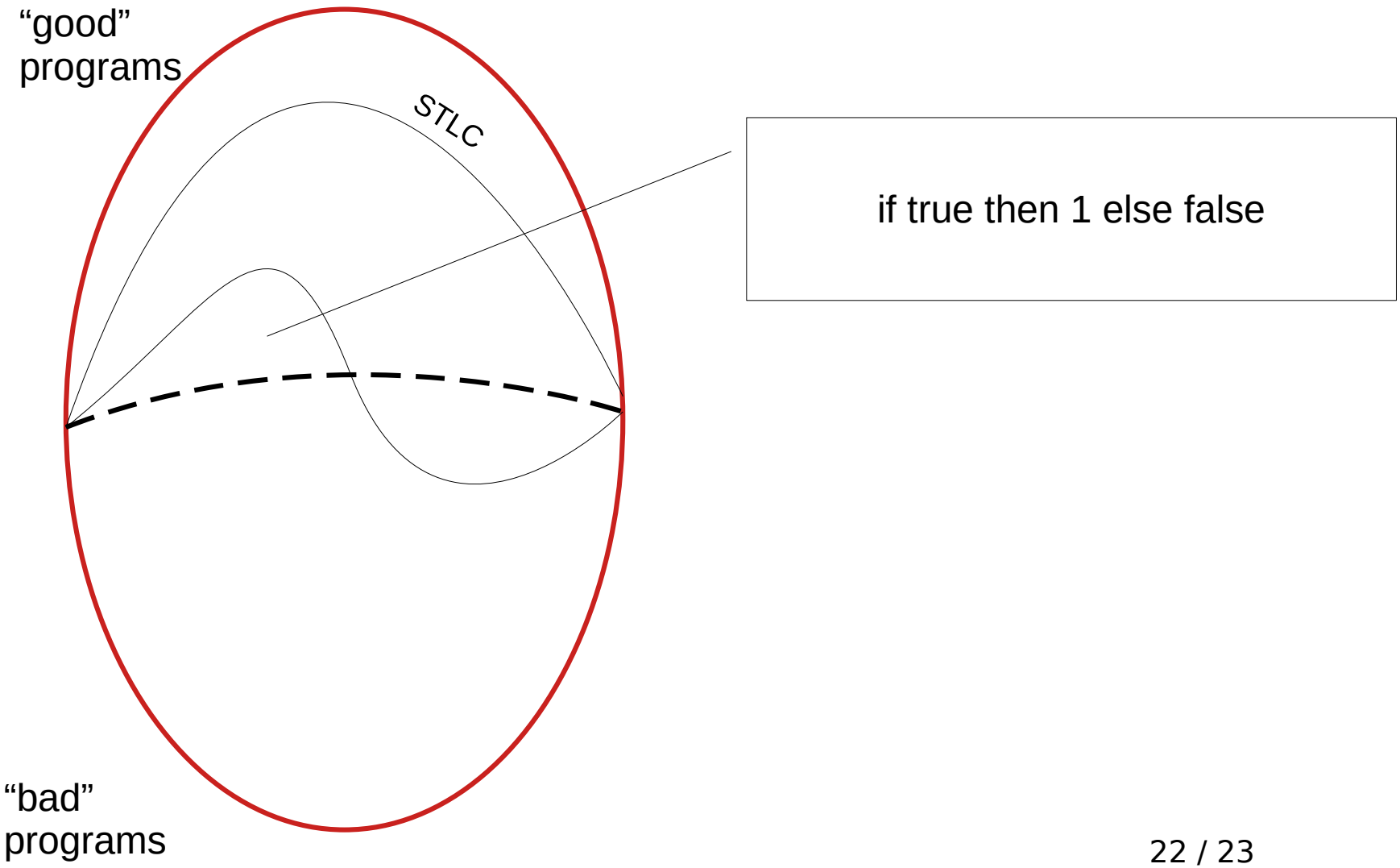
Well-typed terms



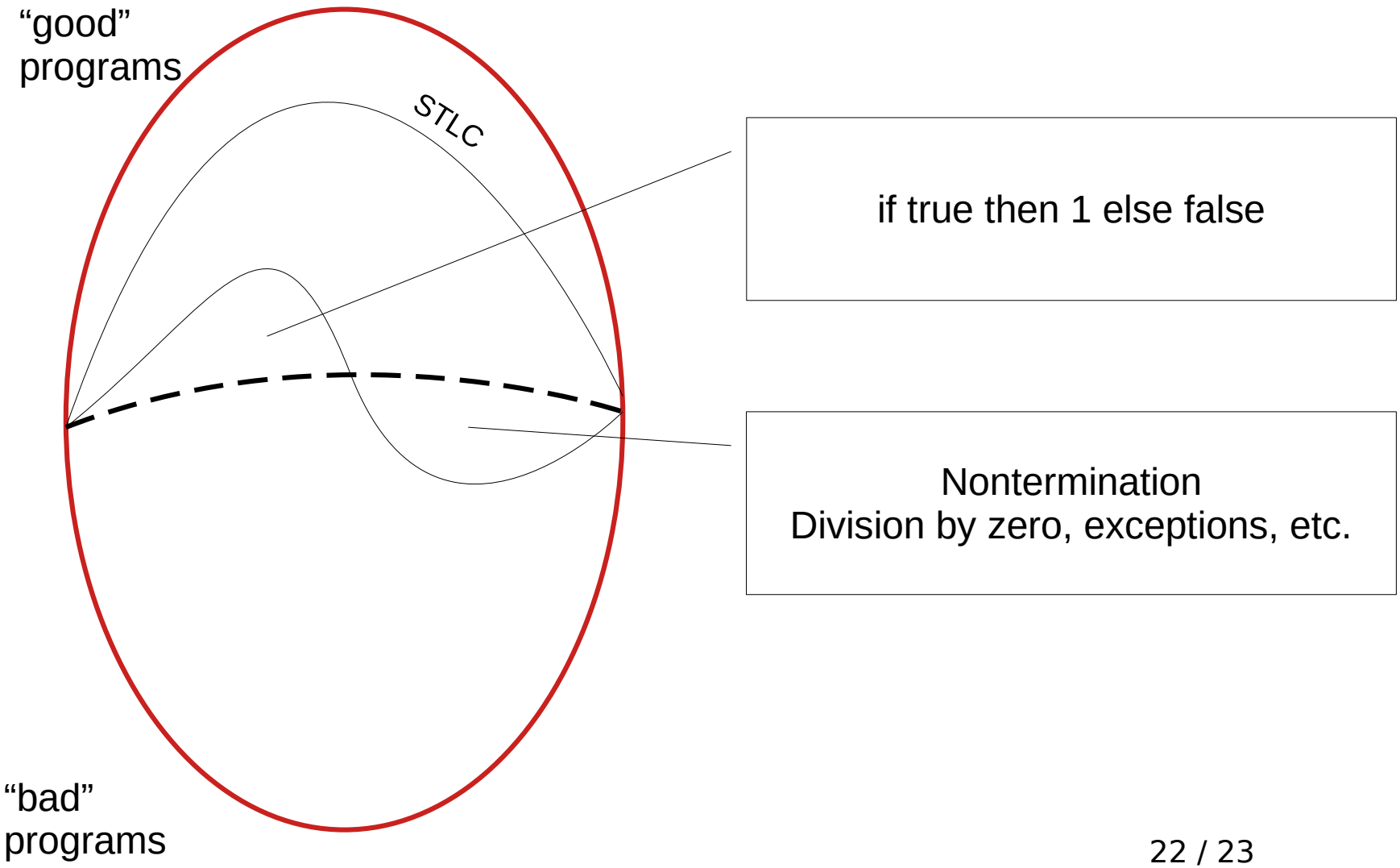
Well-typed terms



Well-typed terms



Well-typed terms



Summary

- Lambda calculus extensions
- Simply Typed Lambda Calculus
  - Statically avoid nonsensical terms
  - Metatheory:

    CR Type soundness (Progress+Preservation)

    Strong normalisation

    Lost: fixed point theorem (thus gen. recursion)

Towards a more realistic language

- convenient to *annotate* types in programs
- we also need a **datatype definition** mechanism so preservation/progress proofs do not need to change with every new kind of data
- examples: OCaml Light, CakeML